Exercise 2

Develop and compare CLI, GUI, and Voice User Interfaces (VUI) for the same ask and assess user satisfaction using Python (Tkinter for GUI, Speech Recognition for VUI), Terminal

AIM:

The aim is to develop and compare Command Line Interface (CLI), Graphical User Interface

(GUI), and Voice User Interface (VUI) for the same task, and assess user satisfaction using

Python (with Tkinter for GUI and Speech Recognition for VUI) and Terminal.

PROCEDURE:

i) CLI (Command Line Interface)

CLI implementation where users can add, view, and remove tasks using the terminal.

CODE:

```
tasks = []


def add_task():
    task = input("Enter task: ")
    tasks.append(task)
    print("Task added.")


def view_tasks():
    if len(tasks) == 0:
        print("No tasks to show.")
    else:
        print("Your tasks:")
```

```
        for i in range(len(tasks)):
            print(i + 1, ".", tasks[i])

def remove_task():
    view_tasks()
    num = int(input("Enter task number to remove: "))
    if num >= 1 and num <= len(tasks):
        removed = tasks.pop(num - 1)
        print("Removed:", removed)
    else:
        print("Invalid task number.")

def main():
    while True:
        print("\n1. Add Task")
        print("2. View Tasks")
        print("3. Remove Task")
        print("4. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            add_task()
        elif choice == "2":
            view_tasks()
        elif choice == "3":
            remove_task()
```

```
        elif choice == "4":
            print("Exiting...")
            break
        else:
            print("Invalid choice.")

main()
```

OUTPUT:

```
Enter your choice: 1
Enter task: Sugar
Task added.

1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter your choice: 1
Enter task: Coffee powder
Task added.

1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter your choice:
```

```
5 -- "C:\Users\SHREYA AMUDHU N R\Downloads\tasks = [].py" "

1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter your choice: 2
No tasks to show.

1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter your choice: 1
Enter task: Milk
Task added.
```

```
4. Exit
Enter your choice: 1
Enter task: Coffee powder
Task added.

1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter your choice: 2
Your tasks:
1 . Milk
2 . Sugar
3 . Coffee powder

1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter your choice:
```

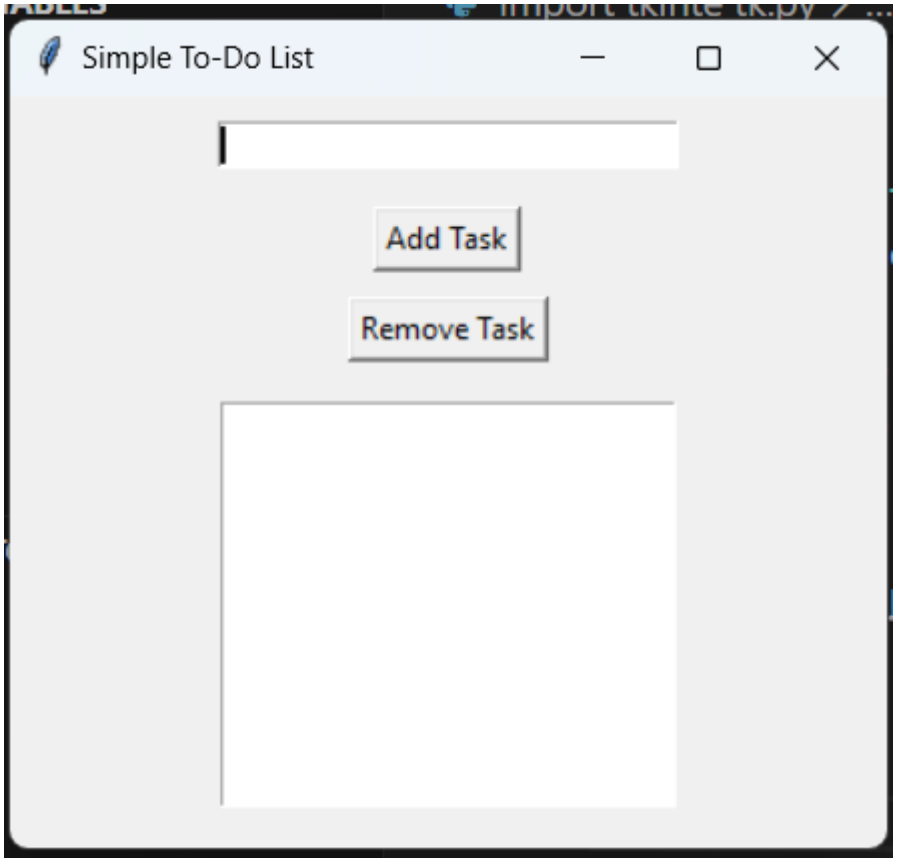
ii) GUI (Graphical User Interface)

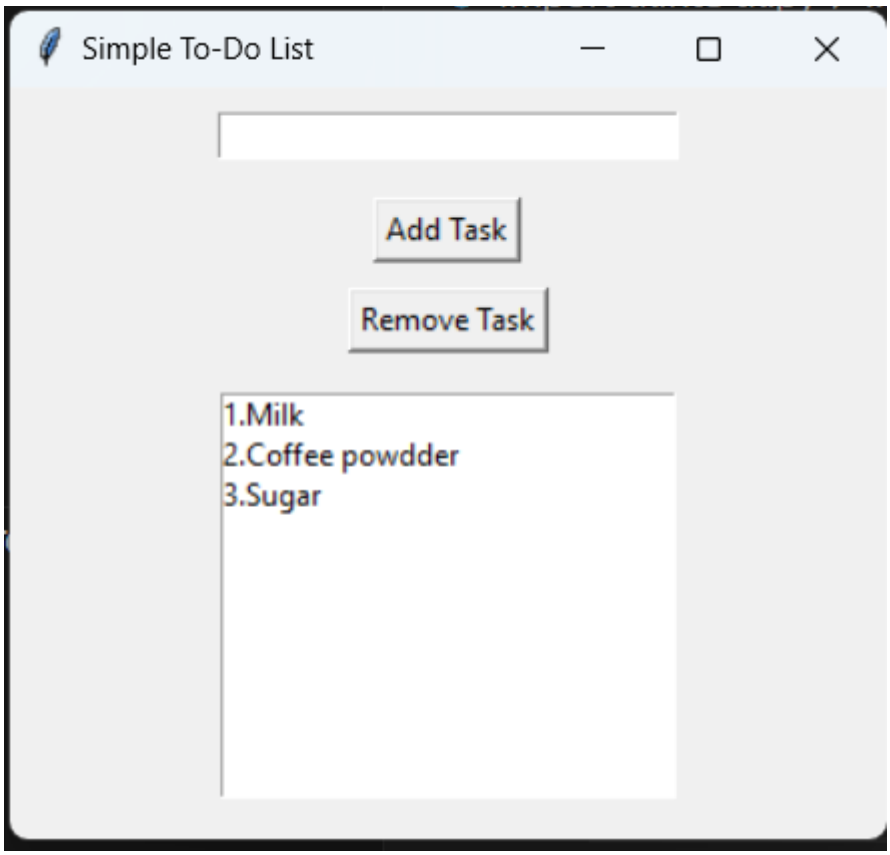
Tkinter to create a simple GUI for our To-Do List application.

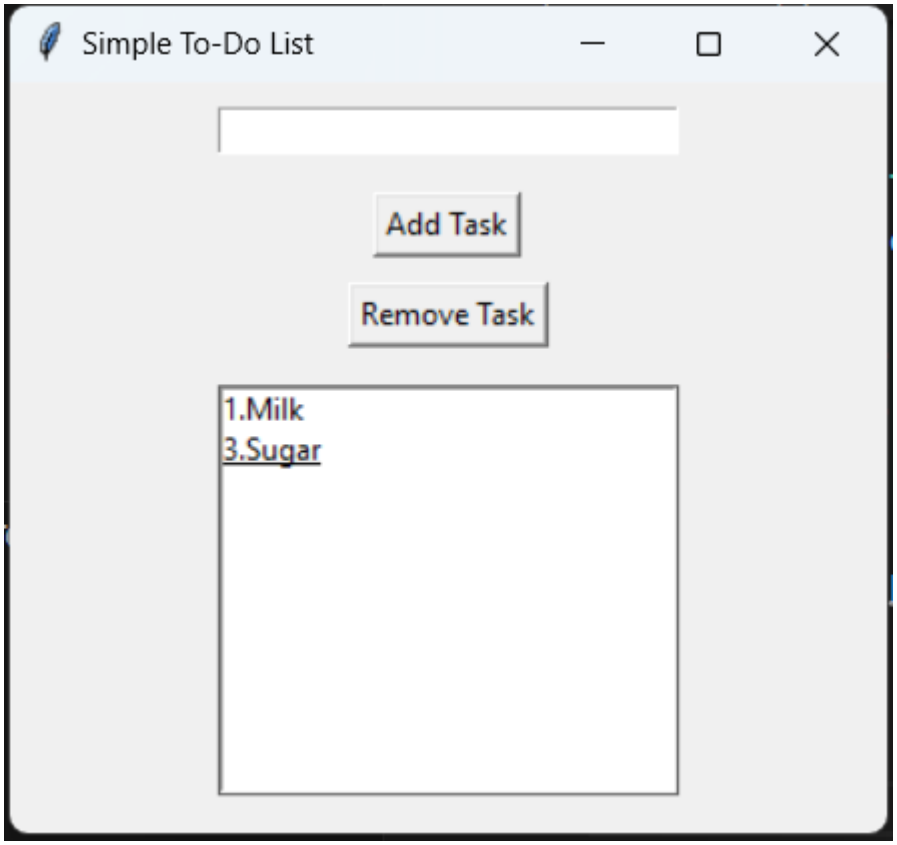
```
import tkinter as tk
from tkinter import messagebox

# Create main window
app = tk.Tk()
app.title("Simple To-Do List")
app.geometry("350x300")

tasks = []

# -------- Functions --------

def add_task():
    task = entry.get().strip()
    if task == "":
        messagebox.showwarning("Warning", "Task cannot be empty!")
        return
    tasks.append(task)
    listbox.insert(tk.END, task)
    entry.delete(0, tk.END)

def remove_task():
    selected = listbox.curselection()
    if not selected:
```

```
        messagebox.showwarning("Warning", "Select a task to remove!")
        return
    index = selected[0]
    listbox.delete(index)
    tasks.pop(index)

# -------- UI Elements --------

entry = tk.Entry(app, width=30)
entry.pack(pady=10)

tk.Button(app, text="Add Task", command=add_task).pack(pady=5)
tk.Button(app, text="Remove Task", command=remove_task).pack(pady=5)

listbox = tk.Listbox(app, width=30, height=10)
listbox.pack(pady=10)

app.mainloop()
```

OUTPUT:


Simple To-Do List
Add Task
Remove Task


Simple To-Do List
Add Task
Remove Task
1.Milk
2.Coffee powdder
3.Sugar


Simple To-Do List
Add Task
Remove Task
1.Milk
3.Sugar

+

iii) VUI (Voice User Interface)

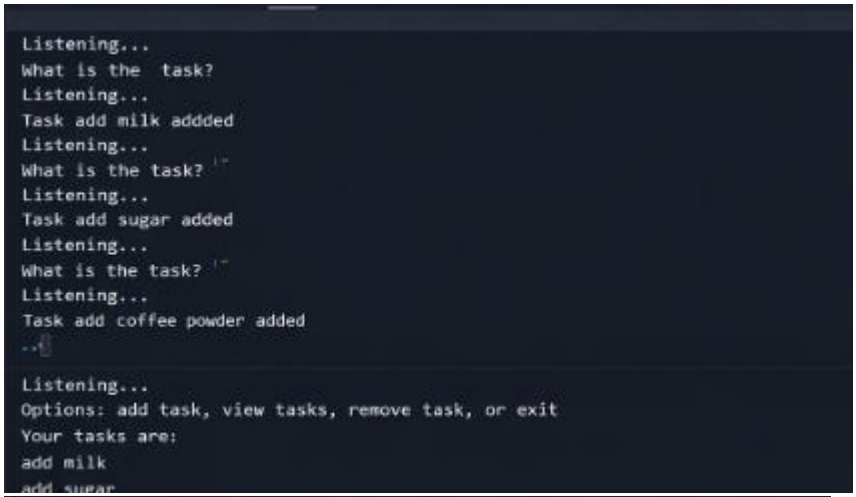
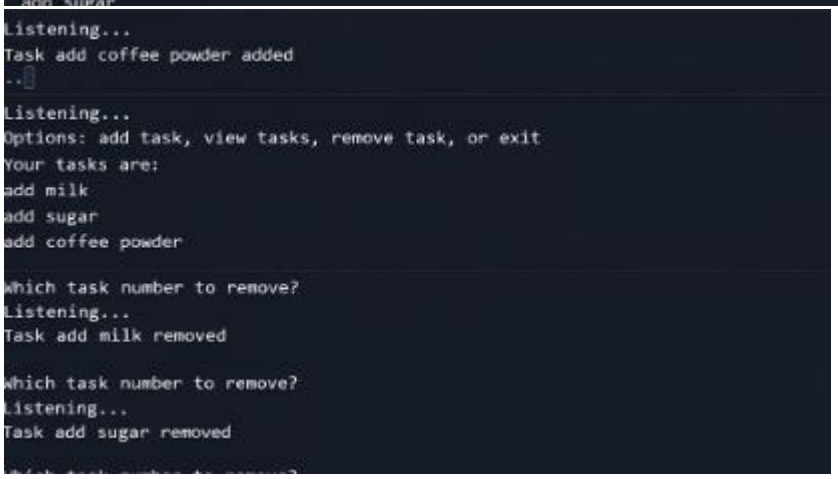
speech_recognition library for voice input and the pyttsx3 library for text-to-speech

output. Make sure you have these libraries installed (pip install SpeechRecognition

pyttsx3).

```
import speech_recognition as sr

import pyttsx3

tasks = []

recognizer = sr.Recognizer()

engine = pyttsx3.init()



def add_task(task):

    tasks.append(task)

    engine.say(f"Task {task} added")

    engine.runAndWait()



def view_tasks():

    if tasks:

        engine.say("Your tasks are")

        for i, task in enumerate(tasks, start=1):

            engine.say(f"{i}. {task}")

    else:

        engine.say("No tasks to show")

    engine.runAndWait()
```

```
def remove_task(task_number):
    if 0 < task_number <= len(tasks):
        removed_task = tasks.pop(task_number - 1)
        engine.say(f"Task {removed_task} removed")
    else:
        engine.say("Invalid task number")
    engine.runAndWait()


def recognize_speech():
    with sr.Microphone() as source:
        print("Listening...")
        audio = recognizer.listen(source)

    try:
        command = recognizer.recognize_google(audio)
        print("You said:", command)
        return command.lower()
    except sr.UnknownValueError:
        engine.say("Sorry, I did not understand that")
        engine.runAndWait()
        return None


def main():
    while True:
```

```
engine.say("Options: add task, view tasks, remove task, or exit")
engine.runAndWait()

command = recognize_speech()

if not command:
    continue

if "add task" in command:
    engine.say("What is the task?")
    engine.runAndWait()
    task = recognize_speech()
    if task:
        add_task(task)

elif "view tasks" in command:
    view_tasks()

elif "remove task" in command:
    engine.say("Which task number to remove?")
    engine.runAndWait()
    task_number = recognize_speech()
    if task_number and task_number.isdigit():
        remove_task(int(task_number))
    else:
        engine.say("Please say a valid number")
        engine.runAndWait()
```

```
        elif "exit" in command:
            engine.say("Exiting...")
            engine.runAndWait()
            break

        else:
            engine.say("Invalid option. Please try again.")
            engine.runAndWait()
if __name__ == "__main__":
    main()
```